

Revision Notes: Module 3

Comprehensive Feature Engineering I (Categorical, Numerical, & Interaction Spaces)

Roman

May 22, 2026

The Strategic Matrix: Categorical Encoding & Cardinality Laws

1. The Core Engineering Logic

Machine learning algorithms are pure numerical calculators; they evaluate feature inputs via geometric distances or optimization gradients. They cannot natively parse text strings. The architectural constraint that dictates your choice of categorical transformation is **Cardinality**—the absolute count of unique string elements within a data column.

2. Structural Implementations

- **One-Hot Encoding (OHE):** Maps an unstructured string column into a series of new parallel binary indicator vectors (0 or 1).
 - *The Cardinality Limit:* Deploy exclusively on **Low Cardinality** features (< 10 unique entries, e.g., `Device_Type`).
 - *The Mathematical Necessity* (`'drop_first=True'`): When encoding a feature with K options, you must drop the first dummy column, leaving $K - 1$ columns. If you keep all K vectors, they will sum up to a perfect vector of 1s, creating an exact linear relationship with your model's intercept term. This causes severe **multi-collinearity** (the dummy variable trap), which destabilizes the matrix inversion steps in linear, logistic, and neural network models.
- **Ordinal Encoding:** Assigns explicit, ordered integers to string values to match an inherent human or physical hierarchy.
 - *The Strategy:* Use when categories follow a clear progression (`None_Used` $\rightarrow 0$, `SAVE10` $\rightarrow 1$, `WELCOME` $\rightarrow 2$, `FLASH20` $\rightarrow 3$). This tells your model's optimization gradients that a value of 3 represents a greater structural impact than 1.
- **Frequency (Count) Encoding:** Replaces every text element with either its total row count or its relative percentage frequency in the dataset.
 - *The Strategy:* The absolute gold standard for **High Cardinality** columns (e.g., thousands of `ZIP_Codes` or `Global_Cities`). It measures the **rarity** or **density** of a category, capturing a massive predictive signal without blowing up your table structure.

3. Real-World Pitfalls & Leaderboard Landmines

1. **The Missing Dictionary Key Trap:** When applying a manual ordinal mapping using Pandas `.map()`, if you miss a newly generated string category (like yesterday's cleaned `'None_Used'` value), Pandas will silently convert that unmatched row into a missing value (NaN). This reintroduces messy gaps into your data pipeline right before training.

2. **The Curse of Dimensionality:** Blindly running OHE on a column with 8,500 cities adds 8,500 columns to your table. Because a single user can only be in one city at a time, 8,499 of those columns will be filled with zeros for every single row. This massive, empty matrix layout causes tree-based models to overfit heavily and quickly exhausts your notebook's available RAM.

Leveling the Metric Field: Numerical Scaling & Skewness

1. The Core Engineering Logic

Features with raw, unscaled numeric scales skew optimization paths. If your **Age** column ranges from 18 to 80 and your **Annual_Income_USD** ranges from \$30,000 to \$1,500,000, distance-based models (KNN, K-Means, SVM) or gradient-driven networks will treat income as thousands of times more important simply because its raw numbers are larger.

2. The Mathematical Formulations

- **StandardScaler (Z-Score):** Reshapes your continuous distribution to center around a mean (μ) of 0.0 with a standard deviation (σ) of 1.0:

$$z = \frac{x - \mu}{\sigma}$$

Use Case: Perfect for features that already follow a relatively symmetric, bell-shaped Gaussian distribution.

- **MinMaxScaler:** Squishes an entire column's data array into a bounded interval between exactly 0.0 and 1.0:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Use Case: Highly critical for neural networks that require stable, bounded input ranges.

- **The Logarithmic Transformation ($\ln(x + 1)$):** Reshapes highly asymmetric, right-skewed columns that feature a long tail of extreme outliers.

$$x_{\log} = \ln(x + 1)$$

The 1 Trick: The raw logarithm of zero is mathematically undefined ($\ln(0) \rightarrow -\infty$). If a customer has a recorded transaction value of \$0.00, a raw log operation will crash your pipeline. We use `np.log1p`, which adds 1 to every value before taking the log to keep the operation safe.

3. Real-World Pitfalls & Leaderboard Landmines

1. **The MinMax Outlier Squash:** If your column contains a massive outlier (like our customer earning \$1,500,000), running **MinMaxScaler** raw will map that outlier to exactly 1.0. Because that value is massive compared to the rest of the data, all your regular customers' salaries get squashed into a tiny range like [0.007, 0.05]. This wipes out the variance your model needs to learn, making your users look virtually identical.
2. **The Tree-Scaling Myth:** Tree-based models (XGBoost, LightGBM, Random Forests) are invariant to scaling because they split features based on value order rather than spatial distances. However, failing to scale is still a huge mistake in a datathon because your winning submission will almost always require blending (ensembling) your tree models with neural networks or distance models, which do require uniform scaling.

1. The Core Engineering Logic

- **Interaction Features:** Sometimes, the most informative predictive signal doesn't live in a single column; it lives in the structural relationship between multiple columns. Explicitly calculating these interactions turns highly complex, non-linear optimization problems into simple, easily traceable linear solutions for your model.
- **Continuous Binning (Quantization):** Continuous features often contain small, random fluctuations that represent nothing but pure noise. Binning groups continuous values into stable categorical buckets, helping your model ignore that background noise and focus on meaningful behavioral thresholds.

2. Operational Approaches

- **Ratios ($\div$):** Normalizes absolute numbers to show relative efficiency or risk metrics (e.g., $\text{Income Velocity} = \frac{\text{Annual Income}}{\text{Age}}$).
- **Products ($\times$):** Captures dependencies where the impact of one feature changes based on the size of another (e.g., $\text{Lot Area} = \text{Frontage} \times \text{Depth}$).
- **Differences ($-$):** Tracks financial gaps or temporal progress metrics ($\text{Profit} = \text{Revenue} - \text{Cost}$).
- **Range Binning (`pd.cut`):** Segments data based on explicit, user-defined numeric boundaries. Perfect for real-world regulatory buckets (e.g., $\text{under } 18 = \text{'Minor'}$).
- **Quantile Binning (`pd.qcut`):** Automatically adjusts category boundaries based on row density, ensuring that each bin contains roughly the exact same number of data samples (e.g., splitting a population into equal Low, Medium, and High spending tiers).

3. Real-World Pitfalls & Model Quantization Connections

1. **The Duplicate Bin Edge Error:** If your data column contains zero variance across a large percentage of its rows (e.g., 90% of users have exactly 1 click), calling `pd.qcut(df['Clicks'], q=4)` will crash your pipeline with a `ValueError`. Because the 25th, 50th, and 75th percentiles are all identical, Pandas cannot mathematically establish unique boundaries to separate the data into equal populations.
2. **The LLM Quantization Bridge (Q4/Q5/Q8):** The underlying mathematical concept behind feature binning is the exact same engine used to run massive large language models locally. Instead of binning input data columns, model quantization takes high-precision **FP16** decimal weights and snaps them onto a small, restricted scale of integer slots (e.g., a 4-bit integer scale for a Q4 model). This strips away subtle weight noise, compresses the model's physical size by 75